# Criterion C: Development

Word Count: 900 (roughly rounded up)

## Table of Contents

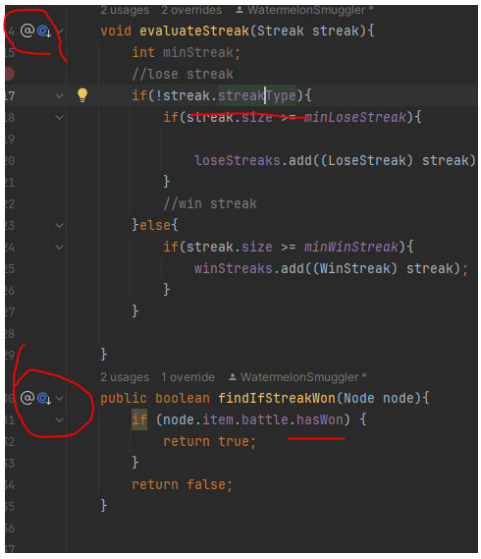
## Using abstract class and interface, and polymorphism in Algorithm package for enhanced modularity

IndividualStatsCalculator and WallAlgorithm in Algorithms package share many similar methods and variables, and I have extended them to abstract class StatsAlgorithm to reduce boilerplate code and simplify future expansions of my program.

```
3 usages  ▲ WatermelonSmuggler
Streak startNewStreak(Node node){
    Streak streak;
    if (findIfStreakWon(node)) {
        streak = new WinStreak();
    } else {
        streak = new LoseStreak();
    }
    //"first" game will always contribute to Streak
    streak.addMode(node.item.event.mode);
    return streak;
}
4 usages  ▲ WatermelonSmuggler
void findStreak(Node node, Streak streak){
    //edge case here:
    if(node == null){
        evaluateStreak(streak);
    }else{
        //if won == won, lose == lose; streak is being maintained?
        if(findIfStreakWon(node) == streak.streakType){
            streak.size++;
            streak.addMode(node.item.event.mode);
            findStreak(node.reference, streak);
        }else{
            evaluateStreak(streak);
            streak = startNewStreak(node);
            //new streak
            findStreak(node.reference, streak);

        }
    }
}
```

In StatsAlgorithm above, the methods StartNewStreak and findStreak are both used normally by the IndividualStatsCalculator and WallAlgorithm. Hence, when change needs to be made, only one method needs to be edited instead of 2 separate copies. It helps greatly with debugging and adding improvements.



```
2 usages  2 overrides  ▲ WatermelonSmuggler *
void evaluateStreak(Streak streak){
    int minStreak;
    //lose streak
    if(!streak.streakType){
        if(streak.size >= minLoseStreak){

            loseStreaks.add((LoseStreak) streak);
        }
        //win streak
    }else{
        if(streak.size >= minWinStreak){
            winStreaks.add((WinStreak) streak);
        }
    }

}
2 usages  1 override  ▲ WatermelonSmuggler *
public boolean findIfStreakWon(Node node){
    if (node.item.battle.hasWon) {
        return true;
    }
    return false;
}
```

On the other hand, the above methods in StatsAlgorithm which are used in the previous 2 recursive methods are overwritten by IndividualStatsCalculator and WallAlgorithm respectively so they can carry out slightly different versions of similar tasks.

To calculate stats of different players for the sall of honor/shame, the WallAlgorithms compose of a collection of IndividualStatsAlgorithm as a container, because they are siblings and do not use the same methods, subclasses of WallAlgorithm would generate an IndividualStatsAlgorithm using their own methods and using a different constructor to insert the data in the IndividualStatsAlgorithm:

```
public WallAlgorithm(BattleLog[] manyItems) {
    this.manyItems = new IndividualStatsCalculator[manyItems.length];
    //Evaluate data and parse them
    for (int i = 0; i < manyItems.length; i++) {
        this.name = manyItems[i].getName();
        BattleLog x = manyItems[i];
        //'first' is automatically overwritten when makeBattle is called
        makeBattle(x.getItems());
        findData(); //findData functional
        findStreak(); //findStreak functional
        setOurGuy(x); //after some ID magickery and .equal forgetting, FUNCTIONAL !!!!
        this.winLoseRate = (double) netWins/ (double) getSize();

        this.manyItems[i] = new IndividualStatsCalculator(winStreaks, loseStreaks, winLoseRate, name);
        this.manyItems[i].playerJudgement = findJudgement(); //FUNCTIONAL (after some lex db config and class adding stuff)

        //LASTLY, REMEMBER TO CLEAR ALL DATA!!!!
        netWins = 0; netTrophy = 0; winLoseRate = 0;
        winStreaks = new LinkedList<>(); loseStreaks = new LinkedList<>();

        System.out.println("created new wall");
    }
}
```

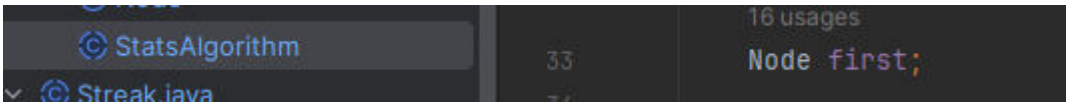
Then:

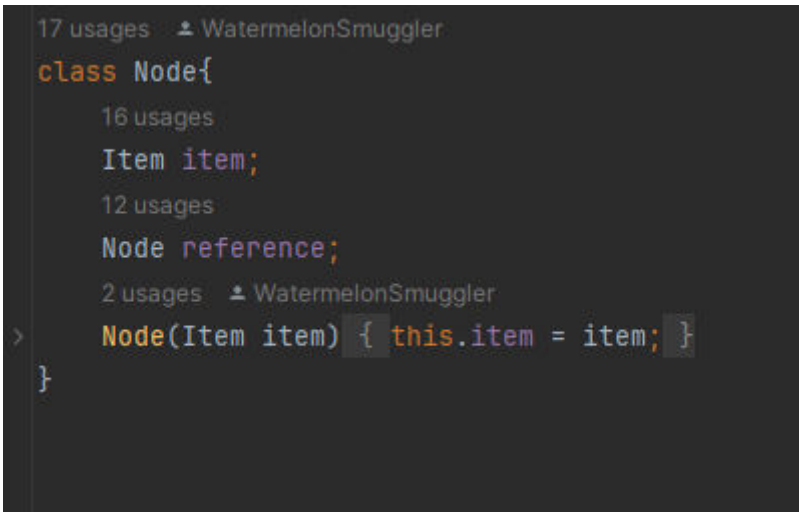
```
IndividualStatsCalculator(LinkedList<WinStreak> winStreaks, LinkedList<LoseStreak> loseStreaks, double winLoseRate, String name){
    this.winStreaks = winStreaks;
    this.loseStreaks = loseStreaks;
    this.winLoseRate = winLoseRate;
    this.name = name;
}

//Constructor for sole purpose of calculating individual stats (no name, runs methods inside constructor
3 usages  WatermelonSmuggler *
public IndividualStatsCalculator(ArrayList<Item> item) {
    System.out.println("new individual stats checker");
    //Gets the Queue of item (which includes battle and event and such) from BattleLog items
    makeBattle(item);
    findData();
    findStreak();
    winLoseRate = (double) netWins/getSize();
}
```

Because most BattleLog calculations depend on streaks, the interface HasStreak helps ensure all current and future calculations classes that use it have a similar structure. Additionally, the 2 final variables extend to all current classes, and editing them would conveniently change the behavior of the whole Algorithm package (Babayev).


## Implementing custom list structure for items Class to recurse through methods



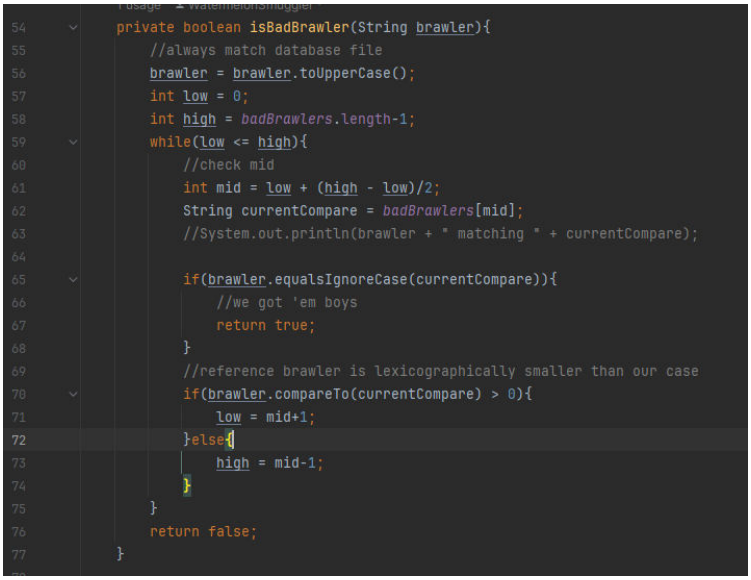

```
void makeBattle(ArrayList<Item> item) {
    //item will be null if no JSON is returned due to API er
    if(item != null) {
        first = new Node(item.get((0)));

        Node reference = first;
        for (int i = 1; i < item.size(); i++) {
            reference.reference = new Node(item.get(i));
            reference = reference.reference;
        }
    }
}
```

Because subclasses of StatsAlgorithm all traverse through a dynamic list of BattleLogs, it contains a custom implementation of a LinkedList to parse my ArrayList<Item>. As such, it also has a Class Node as an inner class. It adds flexibility to my algorithms and provides access to tailored methods that the program may need to analyze data. It's also more intuitive to recursively traverse through battles than to iterate using a for loop and is less prone to errors and strenuous debugging.

Additionally, it is more lightweight compared to the default container as it gets rid of unused methods.

## Using an iterative binary search algorithm to find bad brawler



```
54    private boolean isBadBrawler(String brawler){
55        //always match database file
56        brawler = brawler.toUpperCase();
57        int low = 0;
58        int high = badBrawlers.length-1;
59        while(low <= high){
60            //check mid
61            int mid = low + (high - low)/2;
62            String currentCompare = badBrawlers[mid];
63            //System.out.println(brawler + " matching " + currentCompare);
64
65            if(brawler.equalsIgnoreCase(currentCompare)){
66                //we got 'em boys
67                return true;
68            }
69            //reference brawler is lexicographically smaller than our case
70            if(brawler.compareTo(currentCompare) > 0){
71                low = mid+1;
72            }else{
73                high = mid-1;
74            }
75        }
76        return false;
77    }
```

The WallShame class implments a binary search tree that compares one string against a final array of lexigonally arranged String provided by the database to determine if a player is playing one of the user-defined bad brawlers. While the game has 84 brawlers in total, the UI/Backend allows any string to be parsed through the database due to new brawlers always being introduced. This creates large datasets which calls for a solution with reduced time complexity. It's Log(n) time complexity satisfies my critera while providing rapid calcluation for the Wall of Shame.

## Using Jackson to serialize/deserialize JSON and POJO

The Jackson packages parses JSON into java and vice versa (Saloranta). Because my backend receives JSON from Brawl API and sends JSON of processed data to JavaScript, I need Jackson's Objectmapper class to create "Plain-Old-Java-Objects" (Geekific).

```java
ObjectMapper om = new ObjectMapper();
//ensure Jackson doesn't fail on some stupid things
om.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, state: false);
BrawlAPIAccess api = new BrawlAPIAccess();
//retrieve JSON file from API and deserialize into POJO
BattleLog items = om.readValue(api.getBatteLog(playerID), BattleLog.class);
//instantiate calculation class on POJO, automatically calculates
IndividualStatsCalculator calc = new IndividualStatsCalculator(items.getItems());
//automatically updates the Database with new player if exist
//turn calculation into a POJO for the frontend
IndividualInformation info = calc.getInformation();
```

```java
String JSON = om.writeValueAsString(info);
```

For example, in my IndividualStatsServlet, ObjectMapper is configurated to not fail because it cannot find a POJO variable. Hence, I do not have to create a POJO for all the niche variables of the Brawl API. JSON can also convert any Java Object to a JSON given that the object getters/setters, allowing me to send information to JavaScript.

Jackson‘s ObjectMapper can also access tagged @JsonCreator, allowing for manipulation of POJO data while deserialzing:

```java
@JsonCreator
public Battle(@JsonProperty("result") String result, @JsonProperty("duration") int duration,
              @JsonProperty("rank") int rank, @JsonProperty("trophyChange") int trophyChange,
              @JsonProperty("starPlayer") Player starPlayer, @JsonProperty("teams") ArrayList<ArrayList<Player>> teams) {
    this.result = result;
    //might be bugged, in that case remove int rank in parameter
    this.rank = rank; //initialize rank away from 0
    if (result != null) {
        if (result.equals("victory")) {
            hasWon = true;
        } else {
            hasWon = false;
            if (result.equals("draw"))
                hasDrawn = true;
        }
    } else if(rank > 0 ) {
        if(rank < teams.size()/3){ //scored above 66% of all teams
            hasWon = true;
        }else{
            hasWon = false;
        }

    }else {
        //in cases where there isn't a defined victory condition (very edge case etc: quitting a friendly solo bot match)
        if (trophyChange >= 0) {
            hasWon = true;
        } else {
            hasWon = false;
            if (trophyChange == 0)
                hasDrawn = true;
        }
    }
```

Above, I have efficiently determined whether a player had won a game in the POJO constructor. It is valuable as some objects in the JSON does not state whether a player won with the expected variable.

## Using JavaScript fetch and REST API to send request backend

In order for Frontend to interact with backend, JavaScript's fetch function can send GET requests and process the response afterwards (Fetch API). I referred to GeeksforGeeks' lambda fetch execute cycle to make my own REST API requests when a button is pressed.

Example:

```
1+ usages  new *
function addBadBrawler(){
    const addBad = document.getElementById( elementId: 'addBad').value;
    fetch( input: `/sql/brawler?brawler=${encodeURIComponent(addBad)}` , init: {
        method: 'GET',
        headers: {
            'Accept': 'text/plain'
        }
    }).then()
}
```

Above, I also needed to pass down a parameter, which I learned to do with the encodeURIComponent method (encode…MVN).

## Using JavaScript to make table out of JSON:

I learned how to use JS elements textContent and appendChild to build tables row by row (Working).

```
document.getElementById( elementId: "trophy").innerHTML = data.netTrophy;
document.getElementById( elementId: "wins").innerHTML = data.netWins;
document.getElementById( elementId: "wlr").innerHTML = data.winLoseRate;

const streak = data.streaks;

const tableBody : Element  = document.querySelector( selectors: "#statsTable tbody");
tableBody.innerHTML = "";
streak.forEach(streak => {
    const row : HTMLTableRowElement  = document.createElement( tagName: "tr");
    const cell : HTMLTableCellElement  = document.createElement( tagName: "td")
    cell.textContent = streak;
    row.appendChild(cell);
    tableBody.appendChild(row);
```

A table is necessary for me to present the JSON of the processed data. Using lambda functions to iterate the streaks, the function reterieves the respective String and puts it into a one column table.

In a multi column table, I nested lambda function for rows and its respective cells to iterate the row then its cells like so:

```
.then(data =>{
    document.getElementById( elementId: "numberOfPlayer").innerHTML = data.totalPeople;
    //because it's an ID, statsTable has to be preceded by a #
    const tableBody : Element  = document.querySelector( selectors: "#statsTable tbody");
    tableBody.innerHTML = "";
    data.items.forEach(row => {
        let tr : HTMLTableRowElement  = document.createElement( tagName: "tr");
        row.forEach(cell => {
            let td : HTMLTableCellElement  = document.createElement( tagName: "td");
            td.textContent = cell;
            tr.appendChild(td);
        });

        tableBody.appendChild(tr);
    });
```

## Using SQLite embedded database and Xerial driver to store persistent data

SQLite allows easy transportation of a project database, making my project very lightweight without the need for a standalone server for a DBMS (SQlite). Due to SQLite as a small embedded database, Xerial driver is need to establish connection to the DBMS (sqlite-jdbc).

I found the need to implement an UI for adding ID unnecessary as the people my clients have tags for are all friends that agrees to be put on the walls. Hence, when the IndividualStatsServlet finds a real player, it automatically uses SQLite to add the ID as so:

```
IndividualInformation info = calc.getInformation();

if (!Double.isNaN(info.getWinLoseRate())){ //this guy is real
    try {
        SQLite sql = new SQLite();
        sql.addPlayer(playerID);;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

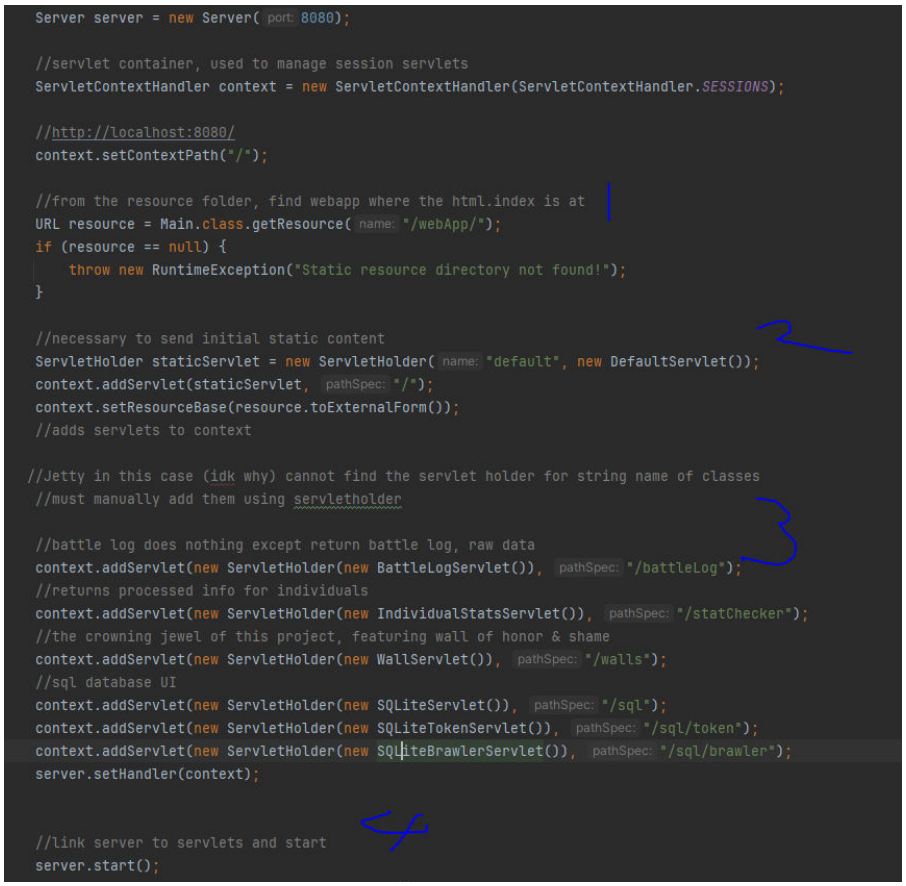
```
2 usages  1 WatermelonSmuggler*
public void addPlayer(String tag) throws SQLException, JsonProcessingException {
    //given: the guy us real
    BrawlAPIAccess api = new BrawlAPIAccess();
    //get his name
    ObjectMapper om = new ObjectMapper();
    om.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, state: false);
    Hello hello = om.readValue(api.getStats(tag), Hello.class);
    st = con.createStatement();
        st.executeUpdate( sql: "INSERT OR IGNORE INTO BrawlerID (nickname, ID) VALUES " +
                "('"+ hello.getName() +"'," +
                "'" + tag + "');");
    System.out.println("attempts to add : " + tag);
    st.close();
}
```

I have also made sure to close all statements once an instsance is no longer needed to ensure that the SQLite database has no open statements that locks future statements from accessing the database.

## Using Jakarta Servlets and Jetty Server to host server using Java

My main method is dedicated to starting and maintaining the servers using the Jetty package (The Eclipse). Jetty makes the process straightforward:

1) Set the Resource folder
2) Make a static servlet and root path
3) Add all the servlets to ContextHandler (Jetty 12).
4) Start server



```
Server server = new Server( port: 8080);

//servlet container, used to manage session servlets
ServletContextHandler context = new ServletContextHandler(ServletContextHandler.SESSIONS);

//http://localhost:8080/
context.setContextPath("/");

//from the resource folder, find webapp where the html.index is at
URL resource = Main.class.getResource( name: "/webApp/");
if (resource == null) {
    throw new RuntimeException("Static resource directory not found!");
}

//necessary to send initial static content
ServletHolder staticServlet = new ServletHolder( name: "default", new DefaultServlet());
context.addServlet(staticServlet, pathSpec: "/");
context.setResourceBase(resource.toExternalForm());
//adds servlets to context

//Jetty in this case (idk why) cannot find the servlet holder for string name of classes
//must manually add them using servletholder

//battle log does nothing except return battle log, raw data
context.addServlet(new ServletHolder(new BattleLogServlet()), pathSpec: "/battleLog");
//returns processed info for individuals
context.addServlet(new ServletHolder(new IndividualStatsServlet()), pathSpec: "/statChecker");
//the crowning jewel of this project, featuring wall of honor & shame
context.addServlet(new ServletHolder(new WallServlet()), pathSpec: "/walls");
//sql database UI
context.addServlet(new ServletHolder(new SQLiteServlet()), pathSpec: "/sql");
context.addServlet(new ServletHolder(new SQLiteTokenServlet()), pathSpec: "/sql/token");
context.addServlet(new ServletHolder(new SQLiteBrawlerServlet()), pathSpec: "/sql/brawler");
server.setHandler(context);


//link server to servlets and start
server.start();
```

For servlets, I used Jakarta EE (Jakarta Servlet) as it meant I no longer need a web.xml since there is a tag that allows me to route the servlets using parameter.

```
@WebServlet("/sql/token")
```